

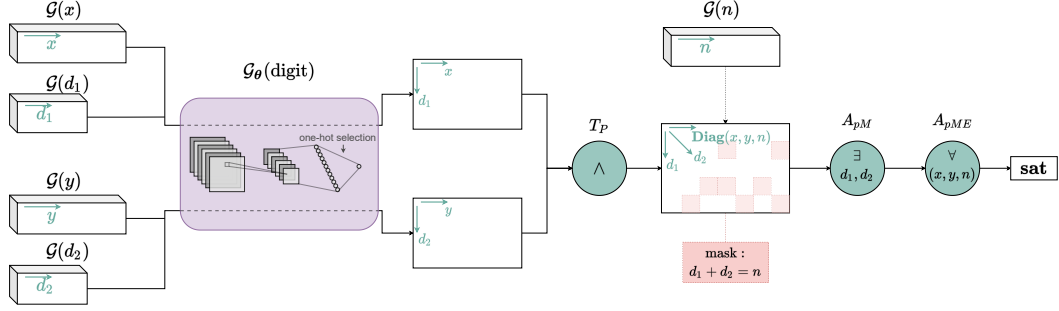


Figure 13: Symbolic Tensor Computational Graph for the Single Digit Addition task. Notice that the figure does not depict accurate dimensions for the tensors; $\mathcal{G}(x)$ and $\mathcal{G}(y)$ are in fact 4D tensors of dimensions $m \times 28 \times 28 \times 1$. Computing results with the variables d_1 or d_2 corresponds to the addition of a further axes of dimension 10.

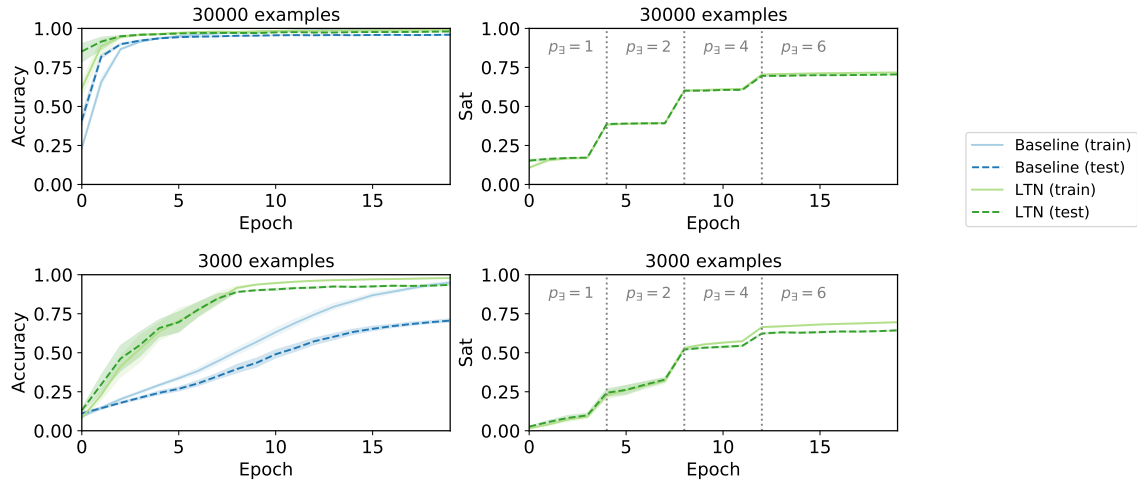


Figure 14: Single Digit Addition Task: Accuracy and satisfiability results (top) and results in the presence of fewer examples (bottom) in comparison with standard Deep Learning using a CNN (blue lines).

Model	(Single Digits)		(Multi Digits)	
	Train	Test	Train	Test
baseline	$2.72 \pm 0.23\text{ms}$	$1.45 \pm 0.21\text{ms}$	$3.87 \pm 0.24\text{ms}$	$2.10 \pm 0.30\text{ms}$
LTN	$5.36 \pm 0.25\text{ms}$	$3.44 \pm 0.39\text{ms}$	$8.51 \pm 0.72\text{ms}$	$5.72 \pm 0.57\text{ms}$

Table 1: The computation time of training and test steps on the single and multiple digit addition tasks, measured on a computer with a single Nvidia Tesla V100 GPU and averaged over 1000 steps. Each step operates on a batch of 32 examples. The computational efficiency of the LTN and the CNN baseline systems are of the same order of magnitude.

Model	Number of training examples			
	(Single Digits)		(Multi Digits)	
	30 000	3 000	15 000	1 500
baseline	95.95 ± 0.27	70.59 ± 1.45	47.19 ± 0.69	2.07 ± 0.12
LTN	98.04 ± 0.13	93.49 ± 0.28	95.37 ± 0.29	88.21 ± 0.63
DeepProbLog	97.20 ± 0.45	92.18 ± 1.57	95.16 ± 1.70	87.21 ± 1.92

Table 2: Accuracy (in %) on the test set: comparison of the final results obtained with LTN and those reported with DeepProbLog[41]. Although it is difficult to compare directly the results over time (the frameworks are implemented in different libraries), while achieving similar computational efficiency as the CNN baseline, LTN also reaches similar accuracy as that reported by DeepProbLog.

4.5. Regression

Another important problem in Machine Learning is regression where a relationship is estimated between one independent variable X and a continuous dependent variable Y . The essence of regression is, therefore, to approximate a function $f(x) = y$ by a function f^* , given examples (x_i, y_i) such that $f(x_i) = y_i$. In LTN one can model a regression task by defining f^* as a learnable function whose parameter values are constrained by data. Additionally, a regression task requires a notion of equality. We, therefore, define the predicate `eq` as a smooth version of the symbol `=` to turn the constraint $f(x_i) = y_i$ into a smooth optimization problem.

In this example, we explore regression using a problem from a real estate data set²⁵ with 414 examples, each described in terms of 6 real-numbered features: the transaction date (converted to a float), the age of the house, the distance to the nearest station, the number of convenience stores in the vicinity, and the latitude and longitude coordinates. The model has to predict the house price per unit area.

Domains:

- `samples`, denoting the houses and their features.
- `prices`, denoting the house prices.

Variables:

- x for the samples.
- y for the prices.
- $\mathbf{D}(x)$ = samples.
- $\mathbf{D}(y)$ = prices.

²⁵<https://www.kaggle.com/quantbruce/real-estate-price-prediction>

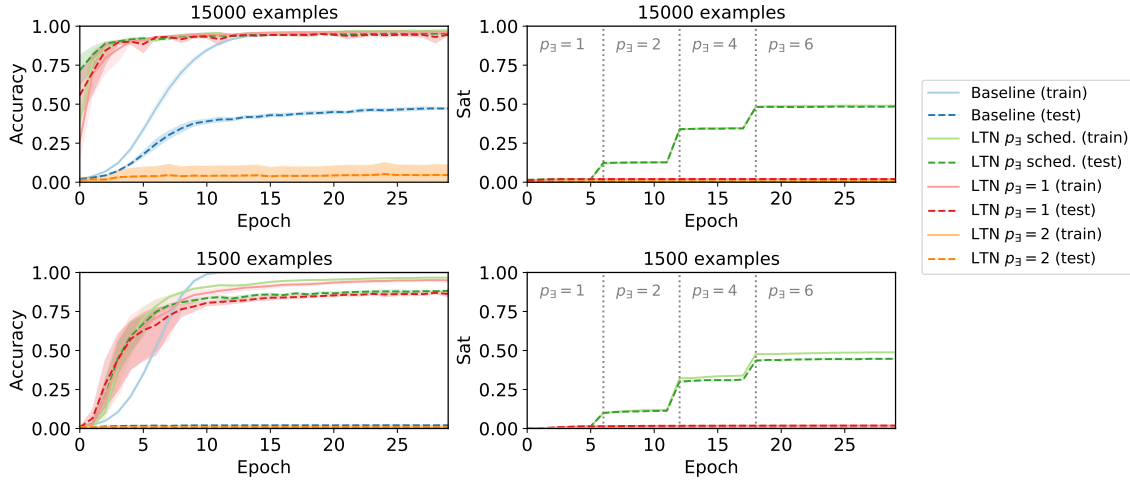


Figure 15: Multiple Digit Addition Task: Accuracy and satisfiability results (top) and results in the presence of fewer examples (bottom) in comparison with standard Deep Learning using a CNN (blue lines).

Functions:

$f^*(x)$, the regression function to be learned.
 $\mathbf{D}_{\text{in}}(f^*) = \text{samples}, \mathbf{D}_{\text{out}}(f^*) = \text{prices}.$

Predicates:

$\text{eq}(y_1, y_2)$, a smooth equality predicate that measures how similar y_1 and y_2 are.
 $\mathbf{D}_{\text{in}}(\text{eq}) = \text{prices}, \text{prices}.$

Axioms:

$$\forall \text{Diag}(x, y) \text{eq}(f^*(x), y) \quad (41)$$

Notice again the use of Diag: when grounding x and y onto sequences of values, this is done by obeying a one-to-one correspondence between the sequences. In other words, we aggregate pairs of corresponding samples and prices, instead of any combination thereof.

Grounding:

$\mathcal{G}(\text{samples}) = \mathbb{R}^6.$
 $\mathcal{G}(\text{prices}) = \mathbb{R}.$
 $\mathcal{G}(x) \in \mathbb{R}^m \times 6, \mathcal{G}(y) \in \mathbb{R}^m \times 1.$ Notice that this specification refers to the same number m of examples for x and y due to the above one-to-one correspondence obtained with the use of Diag.
 $\mathcal{G}(\text{eq}(\mathbf{u}, \mathbf{v})) = \exp(-\alpha \sqrt{\sum_j (u_j - v_j)^2}),$ where the hyper-parameter α is a real number that